Orbit Archive

Unified Cross-Mission Exoplanet Discovery

DBMS Experiential Learning - Level 3 Project

Department of Computer Science and Engineering

Instructor: Dr. Nagasundari S (as named in the supplied requirements)

- Arundhathi K | PES1UG24CS082 | https://github.com/ArundhathiK29
- Bhadra R S | PES1UG24CS111 | https://github.com/be-hope

Section: TO BE PROVIDED

Repository: https://github.com/ArundhathiK29/orbit-archive-project

Abstract

Orbit Archive is a functional cross-mission exoplanet catalogue and research workbench. A relational MySQL database stores stars, planets, observing missions, discoveries and peer validations. MongoDB holds flexible research notes with nested metadata and tag arrays. A Next.js web interface connects catalogue filtering, habitable-zone candidate selection, transactional discovery registration, peer review, automatic audit history, full-text note search and tag aggregation. Database constraints and triggers enforce integrity independently of the interface. All catalogue records are fictional.

Orbit Archive: design and implementation

Problem and objectives

Discovery records from multiple observing missions need a consistent catalogue, reproducible candidate selection, independent peer reviews, and contextual research notes. Orbit Archive connects these workflows while preserving the distinction between a habitable-zone orbital candidate and a confirmed habitable planet. All seed records are fictional teaching examples.

Objectives: unify a cross-mission catalogue; enforce scientifically relevant chronological/relationship rules; expose candidate filtering and aggregate reports; record independent confidence reviews; preserve automatic review history; and search flexible research notes.

Functional requirements

- Authenticate enabled researchers using hashed credentials and signed session cookies.
- Allow Admin and Researcher users to browse/search/filter discoveries and view summary statistics.
- Filter planets using their host star's inner and outer habitable-zone distances.
- Let users create/update/withdraw their own reviews. Derive reviewer identity from the session.
- Let administrators register a planet and its discovery atomically.
- Reject a discovery if the mission did not observe the host star, the mission is planned, or discovery precedes launch.
- Reject reviews before discovery and parent updates that invalidate discovery history.
- Automatically audit review insertion, update and deletion.
- Let authenticated users add and search document-based research notes; aggregate their tags.
- Restrict audit access and registration at the API layer to administrators.

Relational schema

PK denotes primary key; FK denotes foreign key. Complete column types, indexes and checks are in data-dictionary.md.

- star(**star_id PK**, star_name UNIQUE, spectral_type, distance_ly, temperature_k, hz_inner_au, hz_outer_au)
- planet(**planet_id PK**, planet_name UNIQUE, star_id FK -> star, radius_earth, mass_earth, orbital_period_days, orbital_distance_au)
- mission(**mission_id PK**, mission_name UNIQUE, launch_date, agency, status)
- detection_method(**method_id PK**, method_name UNIQUE, description)
- researcher(**researcher_id PK**, first_name, last_name, email UNIQUE, role)
- mission_instrument(**instrument_id PK**, mission_id FK -> mission, instrument_name, instrument_type; UNIQUE(mission_id, instrument_name))
- mission_star(**mission_id PK/FK -> mission, star_id PK/FK -> star**)
- discovery(**discovery_id PK**, planet_id FK -> planet, mission_id FK -> mission, method_id FK -> detection_method, reported_by FK -> researcher, discovery_date, signal_strength, notes; UNIQUE(planet_id, mission_id, method_id,

discovery_date))

- validation(**researcher_id PK/FK -> researcher, discovery_id PK/FK -> discovery**, validation_date, confidence_level)
- validation_audit(**audit_id PK**, researcher_id, discovery_id, action, old_confidence, new_confidence, changed_at)
- app_account(**researcher_id PK/FK -> researcher**, password_hash, enabled)

Relationships and cardinalities

- Star 1 -> 0..many planets; every planet has exactly one host. Binary/circumbinary systems are outside scope.
- Mission 1 -> 0..many instruments; each instrument belongs to one mission.
- Mission many <-> many stars through mission_star; the composite key prevents duplicate observations.
- Planet, mission, detection method, and reporting researcher each have 1 -> 0..many discovery relationships; each discovery has exactly one of each.
- Researcher many <-> many discoveries through validation. A researcher has at most one current review per discovery.
- Researcher 1 -> 0..1 application account.
- Audit identifiers are historical references, deliberately without FKs, so deletion does not erase history.
- MongoDB notes reference SQL discovery/researcher identifiers logically, without a cross-server FK.

Normalization

The core catalogue is designed in 3NF: fields are atomic; mission/star links use a separate junction table; star, mission, method and researcher details are stored once; and review attributes depend on the complete (researcher, discovery) key. Surrogate identifiers and declared natural candidate keys identify entities. The audit is an intentional historical log; author_name in MongoDB is an intentional author-name snapshot, not a normalized SQL relationship.

Why two databases?

MySQL owns structured, related catalogue data and transactional review/discovery operations. Foreign keys, constraints, procedures and triggers enforce relationships and history even when SQL is executed outside the UI.

MongoDB stores narrative observations with arrays of tags and nested, extensible metadata. It supports document validation, text retrieval, and a tag aggregation pipeline without adding catalogue columns whenever a note's context evolves. This is SQL + NoSQL; vector search is not required by this option.

MongoDB model and retrieval

Database: exoplanet_notes. Collection: notes. Required fields: discovery_id, author_id, author_name, title, body, tags, metadata, created_at. Metadata has source and status; additional observational fields can be added. JSON schema validates types, title/body lengths, unique lowercase tags, and status values. The API checks the SQL discovery before insert and sets the author from the signed-in account.

Text index: title (weight 3) and body (weight 1). Search uses MongoDB `$text`, with newest-first display, up to 100 notes. This is lexical search, not semantic similarity. Compound index (discovery_id ASC, created_at DESC) supports discovery note retrieval. Tag analysis runs `$unwind` -> `$group` -> `$sort` -> `$limit` in MongoDB. See scripts/setup.mjs and app/api/stats/route.js.

Advanced SQL concepts used by the application

Concept	Implementation	Visible effect
Transactions	sp_register_discovery + exception handler	Planet and discovery commit together; invalid discovery leaves no partial planet
Views	v_planet_catalogue, v_star_planet_count	Joined catalogue, candidate flag and star counts
Procedures/functions	sp_check_discovery, sp_register_discovery, sp_habitable_zone_candidates, fn_hz_candidate	Shared business rules, atomic registration and SQL candidate demonstration
Triggers	7 original triggers + 4 parent-integrity guards	Valid chronology, observation membership and automatic audit
Indexing	Unique, foreign-key, date, validation and text indexes	Efficient lookup and integrity
Full-text search	ft_discovery_notes	SQL natural-language search demonstrated in 03_queries.sql

The demo also shows explicit SAVEPOINT and ROLLBACK. Views and triggers operate during normal UI/API requests; the SQL demo exposes the function and savepoint directly.

Backend and security

Next.js Route Handlers provide /api/session, /catalogue, /stats, /discoveries, /validations, /notes, /audit. All data endpoints require a valid signed session. Account enablement and role are reloaded from SQL. Passwords use per-account random salts and scrypt; no plaintext passwords are stored in SQL. Session signatures use HMAC-SHA256 with an environment secret, an eight-hour expiry, HttpOnly/SameSite=Lax cookies and optional Secure flag. Mutations require the configured APP_URL origin. SQL values use placeholders, and client input cannot supply author/reviewer identity. Login attempts are limited per email in process memory.

Consistency and limitations

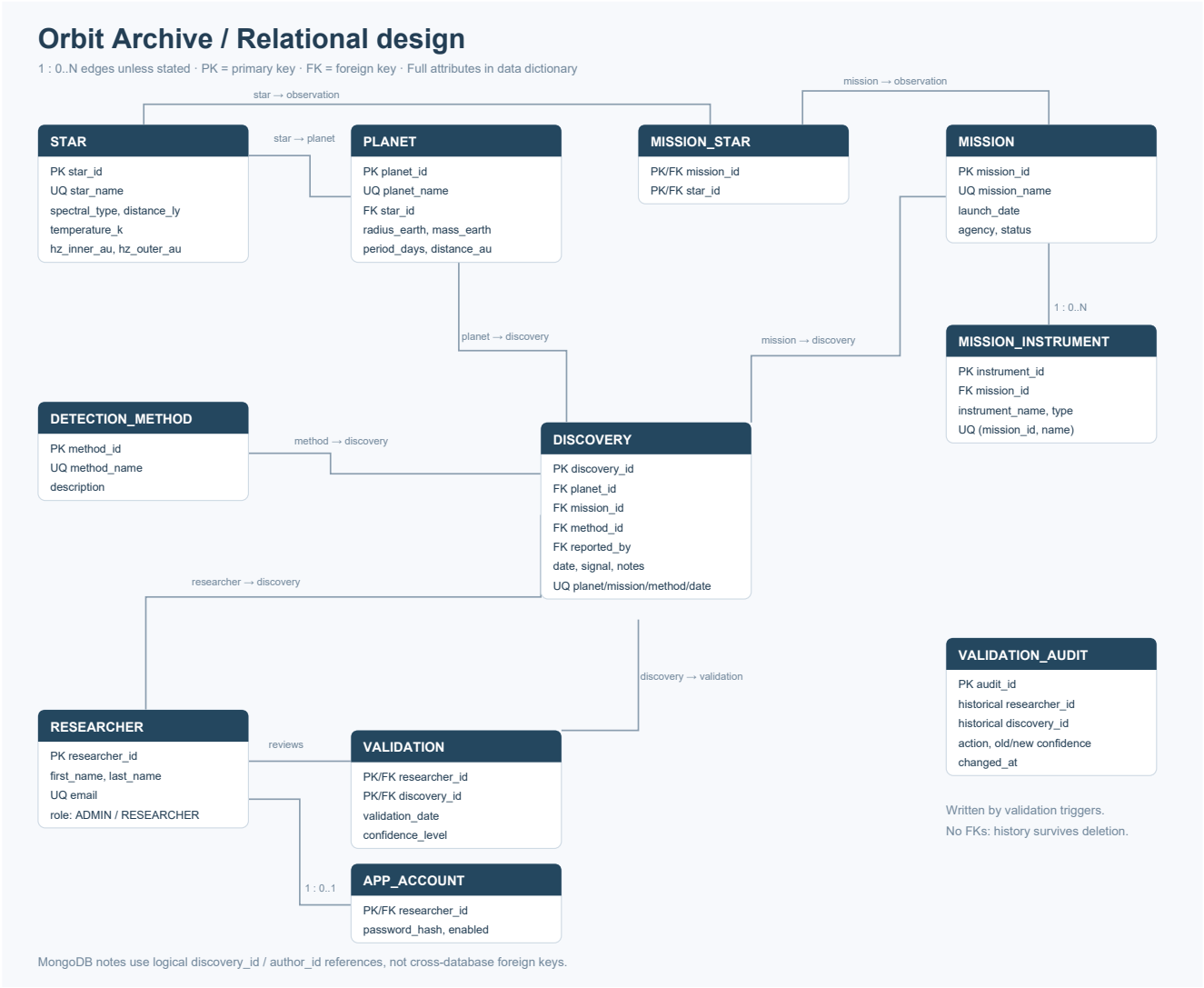
MySQL is the source of truth for catalogue IDs. There is no distributed transaction across MySQL and MongoDB. Note creation checks existence, but external/manual discovery deletion must clean corresponding notes and account for concurrent writes. The UI does not expose discovery deletion. MySQL cascades do not execute child delete triggers; explicitly delete validations when their deletion must be audited. Audit history is retained after disposable test records are removed.

The isolated local demo databases bind to loopback and use no database authentication. Use authenticated, least-privilege database accounts and HTTPS for deployment. The app is intended for one server; shared rate limits/session revocation and distributed consistency are future work. Catalogue/note/audit responses are bounded, not paginated. Astronomical data is synthetic and homogeneous; scientific conclusions are outside this demonstration.

References

- Next.js Route Handlers: https://nextjs.org/docs/app/getting-started/route-handlers
- MongoDB schema validation: https://www.mongodb.com/docs/manual/core/schema-validation/
- MySQL Reference Manual: https://dev.mysql.com/doc/refman/8.0/en/

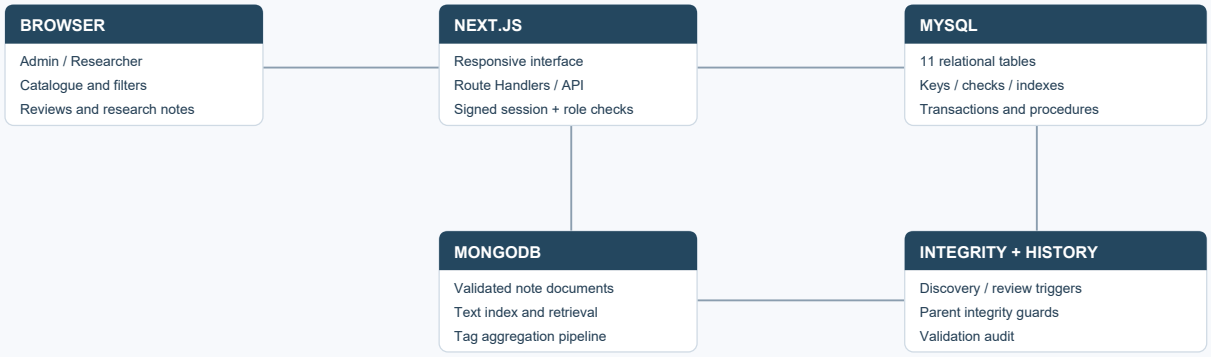
Database and system diagrams



er-diagram.svg

Orbit Archive / System architecture

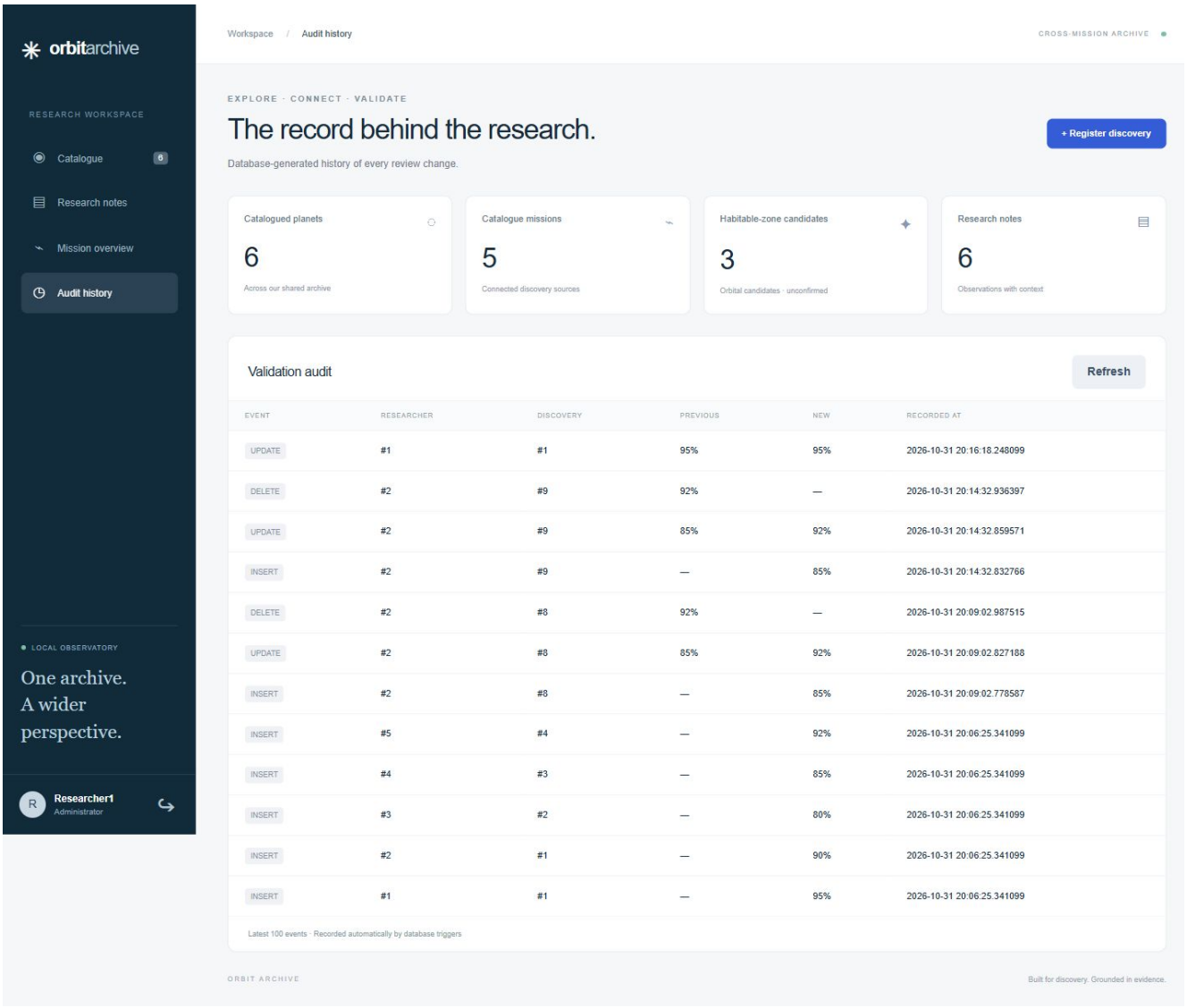
Next.js frontend + backend · MySQL catalogue source of truth · MongoDB research context



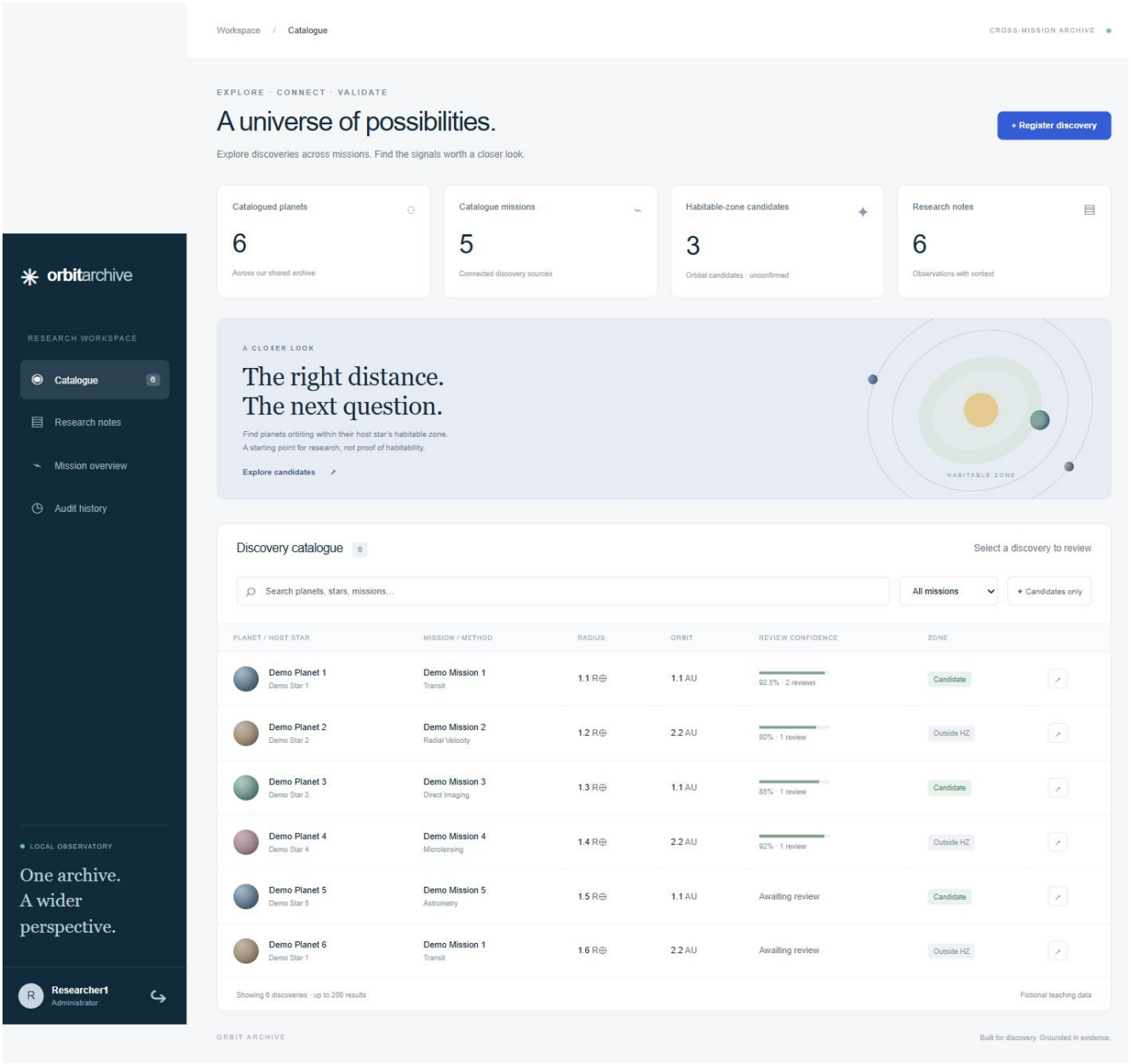
Environment variables supply credentials and the session secret. SQL values use parameter placeholders.

No distributed transaction: SQL discovery existence is checked before note creation; manual deletes require coordinated cleanup.

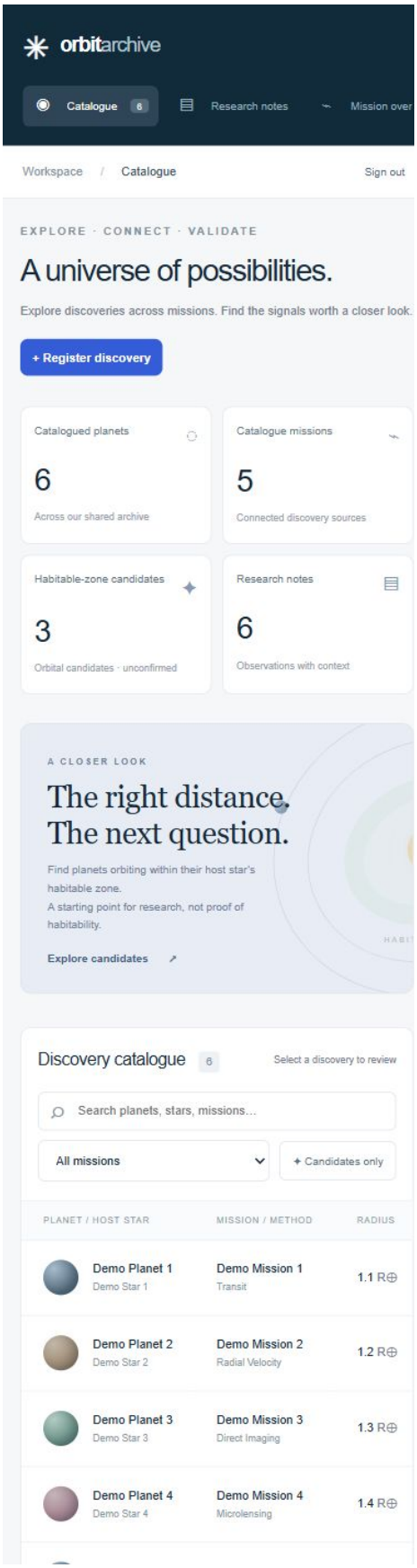
architecture.svg



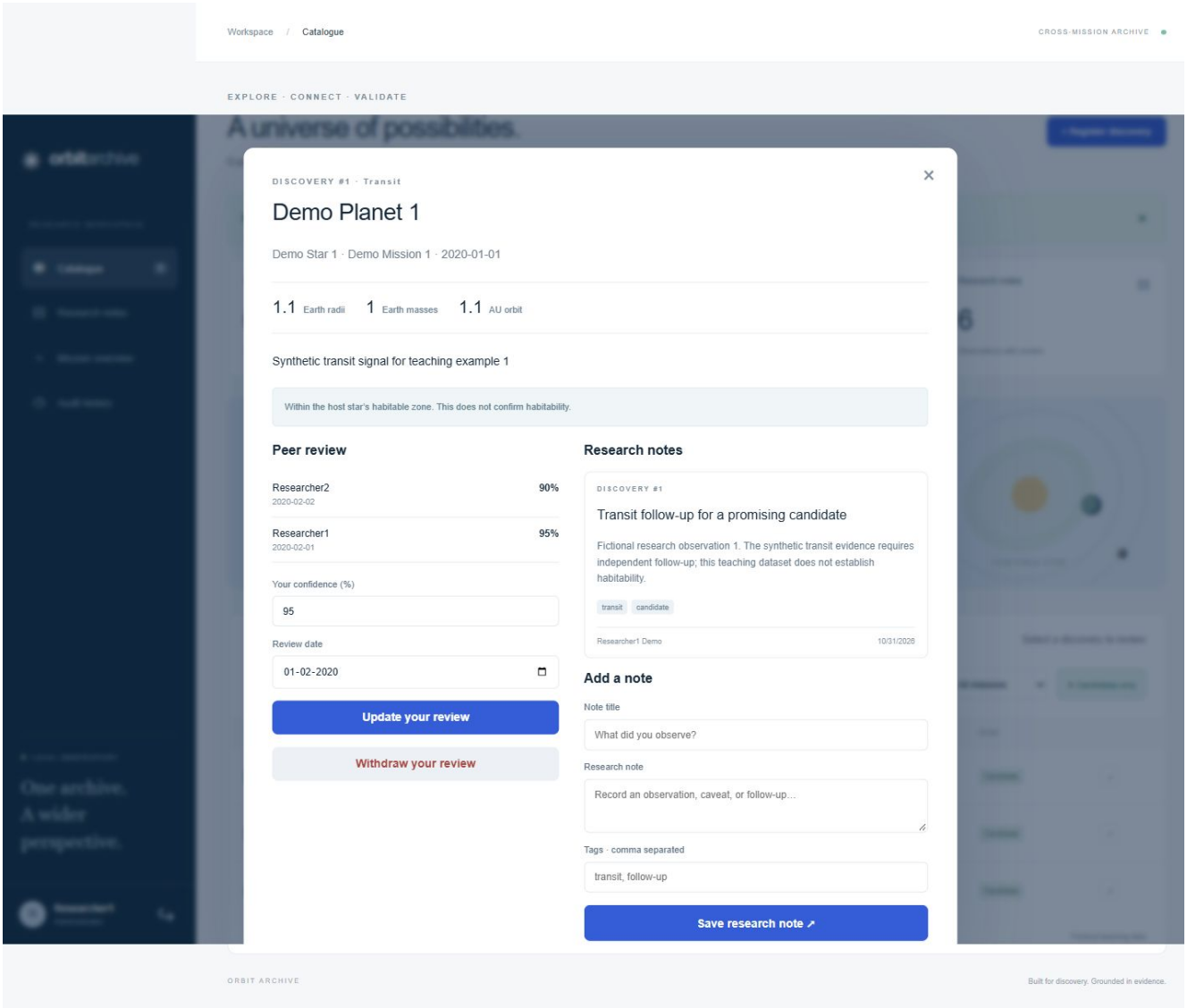
audit history - actual browser capture



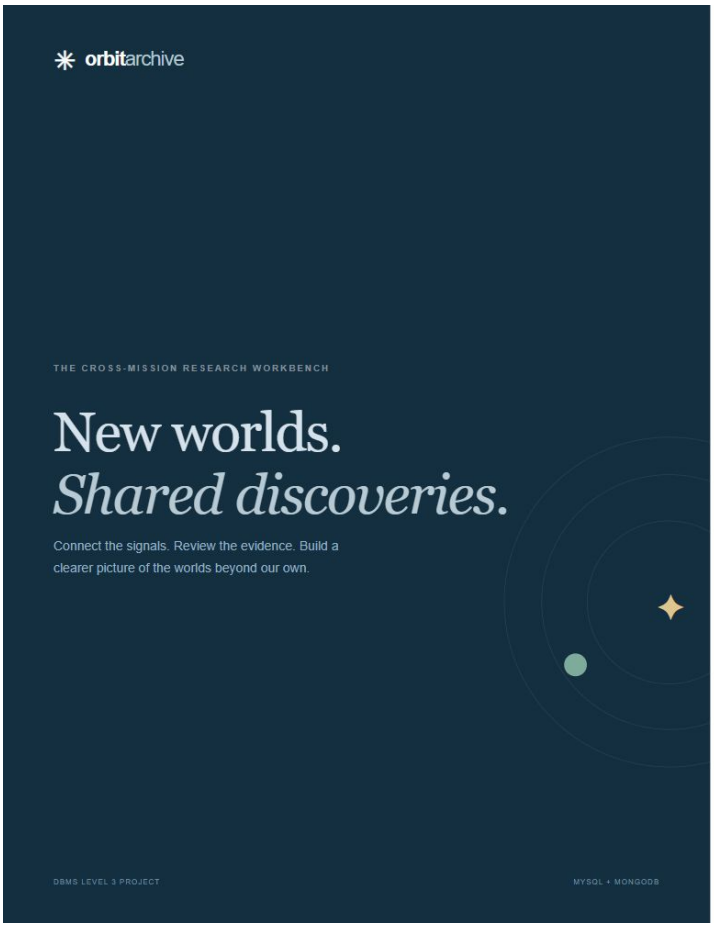
catalogue desktop - actual browser capture



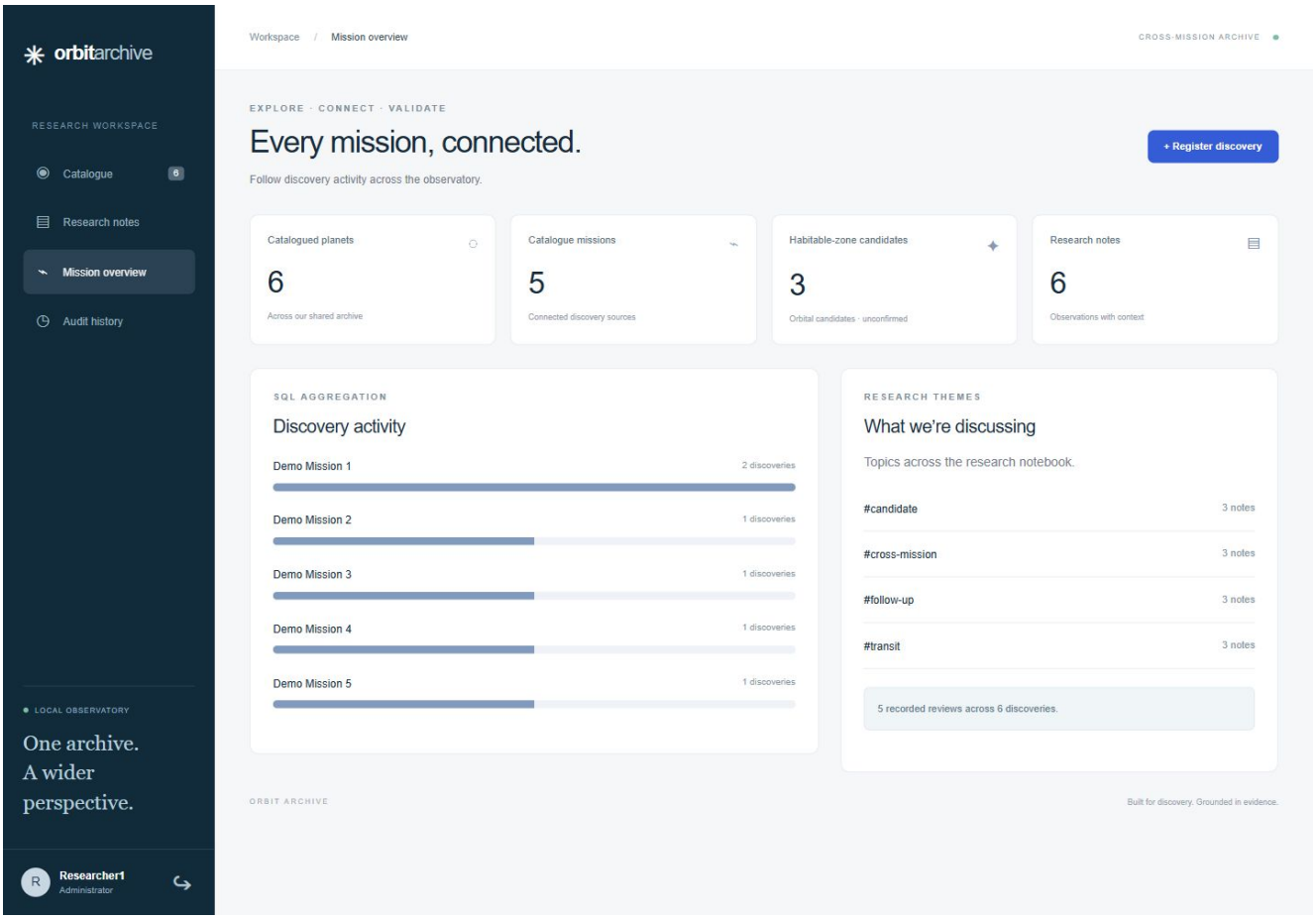
catalogue mobile - actual browser capture



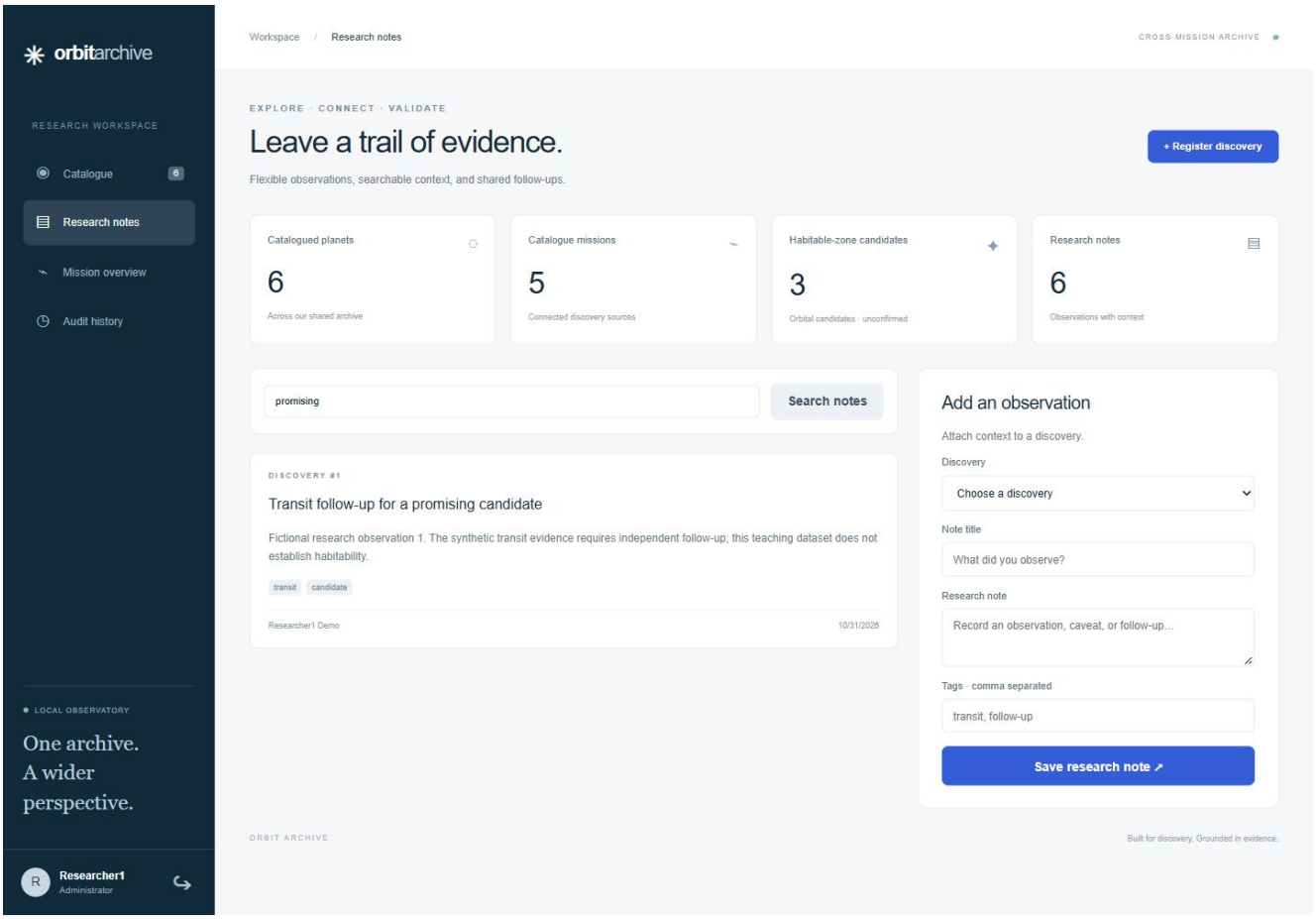
discovery review - actual browser capture



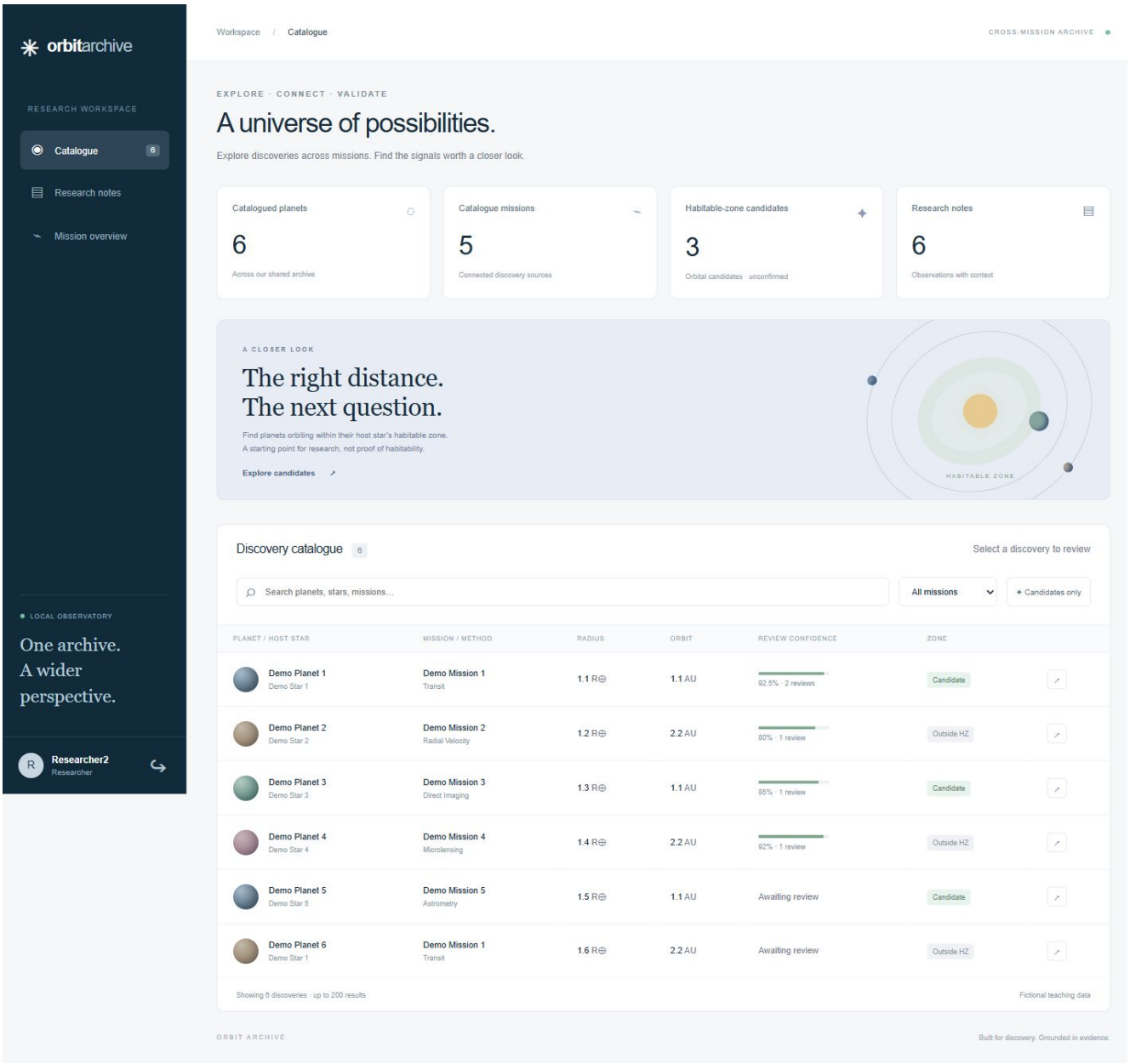
login - actual browser capture



mission overview - actual browser capture



research note search - actual browser capture



researcher role - actual browser capture

Testing and results

The results below come from executable checks, not estimated outcomes.

- SQL: 13/13 passed.
- SQL integrity guards: 4/4 passed.
- MongoDB: 4/4 passed.
- API integration: 34/34 passed.
- Clean SQL installation: 15/15 passed.

Database execution: 2026-10-31T16:12:48.090Z.

API execution: 2026-10-31T16:13:31.561Z.

MySQL server: 26.7.0.

SQL checks

Check	Result
Seed: six planets	PASS
Function: inside zone	PASS
View: zero-planet star	PASS
CHECK rejects negative mass	PASS
CHECK rejects malformed email	PASS
Foreign key rejects planet without host star	PASS
Primary key rejects duplicate validation	PASS
Trigger rejects validation before discovery	PASS
Trigger rejects unobserved host star	PASS
Audit trigger records update	PASS
Rollback restores confidence	PASS
Failed procedure leaves no planet	PASS
Procedure commits discovery	PASS

SQL integrity guards checks

Check	Result
Cannot move a discovered planet to another host	PASS
Cannot move mission launch after discovery	PASS
Cannot remove an observation used by discoveries	PASS
Cannot reassign an observation used by discoveries	PASS

MongoDB checks

Check	Result
Document schema rejects malformed note	PASS
Text and discovery indexes exist	PASS
Full-text retrieval returns seeded notes	PASS
Aggregation returns tag counts	PASS

API integration checks

Check	Result
Unauthenticated catalogue rejected	PASS
Anonymous session is empty	PASS
Invalid password rejected	PASS
Cross-origin login rejected	PASS
Admin can sign in	PASS
Admin session resolves current database role	PASS
Researcher can sign in	PASS
Researcher cannot inspect audit	PASS
Researcher cannot register discovery	PASS
Catalogue returns seeded discoveries	PASS
Candidate filter uses host habitable zone	PASS
Mission filter returns matching source	PASS
Search handles SQL injection text as data	PASS
Invalid measurements rejected	PASS
Unobserved host rejected by SQL trigger	PASS
Rejected registration rolls back planet insert	PASS
Admin registers transactional discovery	PASS
Researcher creates review	PASS
Reviewer identity comes from session	PASS
Out-of-range confidence rejected	PASS
Review before discovery rejected	PASS
Researcher updates own review	PASS
Admin cannot withdraw another reviewer's record	PASS
Researcher creates validated MongoDB note	PASS
Research note stores session author and nested metadata	PASS
MongoDB full-text note search works	PASS
Invalid note tags rejected	PASS
Cross-database discovery reference checked	PASS
Dashboard combines SQL and MongoDB aggregation	PASS
Researcher withdraws own review	PASS

Check	Result
Audit records INSERT, UPDATE and DELETE	PASS
Cross-origin mutation rejected	PASS
Tampered session rejected	PASS
Sign out expires session cookie	PASS

Clean SQL installation checks

Check	Result
Seed: six planets	PASS
Function: inside zone	PASS
View: zero-planet star	PASS
CHECK rejects negative mass	PASS
CHECK rejects malformed email	PASS
Foreign key rejects planet without host star	PASS
Primary key rejects duplicate validation	PASS
Trigger rejects validation before discovery	PASS
Trigger rejects unobserved host star	PASS
Audit trigger records update	PASS
Rollback restores confidence	PASS
Failed procedure leaves no planet	PASS
Procedure commits discovery	PASS
Clean schema contains exactly the 11 intended tables	PASS
No researcher contact-number column in clean schema	PASS

Browser verification

- Administrator and Researcher sign-in verified in browser; sign-out returns to login.
- Catalogue shows 6 planets, 5 missions, 3 candidates and 6 research notes.
- Candidate action filters to 3 matching discoveries.
- Review date before discovery rejected by SQL trigger and shown in dialog.
- Valid review update displays confirmation.
- MongoDB note search for promising returns the matching title.

- Mission overview shows per-mission SQL counts and MongoDB tag aggregation.
- Administrator audit screen displays automatic INSERT, UPDATE and DELETE history.
- Researcher interface hides registration and audit controls.
- Modal Escape closes the dialog and restores focus.
- Invalid note tags preserve title, body and selected discovery.
- Mobile viewport 390x844: document width 375px including scrollbar behaviour; no horizontal page overflow. Catalogue table scrolls within its panel.
- Desktop layout verified at 1440x1000.
- Initial dashboard rendering error was corrected and rechecked; no new console errors observed during final checks.

Important query examples

```
USE exoplanet_db;
-- Q1: JOIN and ORDER BY: cross-mission catalogue.
SELECT d.discovery_id,p.planet_name,s.star_name,m.mission_name,dm.method_name,d.discovery_date
FROM discovery d JOIN planet p USING(planet_id) JOIN star s USING(star_id)
JOIN mission m USING(mission_id) JOIN detection_method dm USING(method_id)
ORDER BY d.discovery_date,d.discovery_id;
-- Q2: GROUP BY, HAVING and aggregates; seed returns Mission 1.
SELECT m.mission_name,COUNT(*) discoveries FROM discovery d JOIN mission m USING(mission_id)
GROUP BY m.mission_id,m.mission_name HAVING COUNT(*) >= 2 ORDER BY discoveries DESC;
-- Q3: Subquery: planets above average mass.
SELECT planet_name,mass_earth FROM planet WHERE mass_earth > (SELECT AVG(mass_earth) FROM planet);
-- Q4: Include stars with zero planets.
SELECT s.star_name,COUNT(p.planet_id) planet_count FROM star s LEFT JOIN planet p USING(star_id)
GROUP BY s.star_id,s.star_name ORDER BY planet_count DESC,s.star_name;
-- Q5: Discoveries without validation (NOT EXISTS).
SELECT d.discovery_id FROM discovery d WHERE NOT EXISTS
(SELECT 1 FROM validation v WHERE v.discovery_id=d.discovery_id);
-- Q6: Average confidence per discovery.
SELECT d.discovery_id,COUNT(v.researcher_id) reviewers,AVG(v.confidence_level) avg_confidence
FROM discovery d LEFT JOIN validation v USING(discovery_id) GROUP BY d.discovery_id;
-- Q7: Full-text search.
SELECT discovery_id,notes FROM discovery
WHERE MATCH(notes) AGAINST('transit' IN NATURAL LANGUAGE MODE);
-- Q8: Meaningful derived candidate filter, not proof of habitability.
SELECT p.planet_name,s.star_name FROM planet p JOIN star s USING(star_id)
WHERE p.orbital_distance_au BETWEEN s.hz_inner_au AND s.hz_outer_au;
-- Q9: Nonpersistent INSERT/UPDATE/DELETE demonstration.
START TRANSACTION;
INSERT INTO researcher(first_name,last_name,email,role)
VALUES('Temporary','Demo','temporary@example.org','RESEARCHER');
SET @demo_researcher=LAST_INSERT_ID();
UPDATE researcher SET first_name='Updated' WHERE researcher_id=@demo_researcher;
SELECT * FROM researcher WHERE researcher_id=@demo_researcher;
DELETE FROM researcher WHERE researcher_id=@demo_researcher;
ROLLBACK;
-- Q10: DDL ALTER/DROP demonstration on a disposable temporary table.
CREATE TEMPORARY TABLE ddl_demo(id INT PRIMARY KEY);
ALTER TABLE ddl_demo ADD COLUMN description VARCHAR(50);
DROP TEMPORARY TABLE ddl_demo;
```

Individual contributions

The following fields must reflect actual team work and commit evidence. Generated implementation is not evidence that both students contributed.

- Arundhathi K: TO BE VERIFIED BY TEAM

- Bhadra R S: TO BE VERIFIED BY TEAM

Conclusion

The implemented system demonstrates a purposeful SQL + NoSQL design with a working Next.js interface. Relational integrity and transactional registration connect database concepts to visible workflows; MongoDB supplies validated, searchable research context and tag analytics. The project goes beyond basic CRUD through candidate classification, independent review, audit history, cross-mission reporting and rollback-based integrity. Real astronomical data import, multi-star systems, pagination, shared rate limits and distributed consistency remain future enhancements.

Submission checklist and demonstration

Implemented

- [x] SQL schema, seed data and meaningful queries.
- [x] Transactions, views, stored routines, triggers, indexes and full-text SQL demonstration.
- [x] MongoDB validated documents, nested metadata, text search and aggregation.
- [x] Next.js interface and seven backend routes.
- [x] Admin and Researcher authentication/authorization.
- [x] Database integrity checks and repeatable API tests.
- [x] ER/architecture diagrams, relational schema, generated data dictionary and report.
- [x] Setup, run and verification commands in README.

Team-specific submission work

- [x] Both team members' names, USNs and GitHub profiles entered in docs/team-details.json.
- [ ] Enter the verified section in docs/team-details.json.
- [ ] Verify title uniqueness and instructor approval for the two-member team.
- [ ] Fill actual responsibilities and contributions; both members must contribute to database and application work.
- [ ] Maintain genuine individual Git commits. Do not invent past contributions or commit authors.
- [x] Public source repository: https://github.com/ArundhathiK29/orbit-archive-project.
- [ ] Run npm run docs:report after team details change.
- [ ] Check your instructor's deadline and final submission format; neither is stated in the supplied PDF.

Demonstration sequence (about 8-10 minutes)

- Explain the cross-mission catalogue/review/context problem, synthetic data and choice of MySQL + MongoDB.
- Show docs/er-diagram.svg and relational schema; explain composite validation key and observation junction.
- Sign in as Researcher. Browse/search catalogue, select Mission 2, then clear filters and choose Candidates only.
- Open Demo Planet 1. Explain radius/orbit and that a habitable-zone flag is not confirmation of habitability.
- Submit/update your confidence review. An invalid date before discovery is rejected by SQL.
- Add a tagged research note; search a distinctive word in Research notes. Open Mission overview and explain the tag aggregation pipeline.
- Sign in as Admin. Register a discovery for Demo Star 1 / Demo Mission 1. An invalid Mission 2/Star 1 combination is rejected; no partial planet remains.
- Inspect Audit history and explain automatic INSERT/UPDATE/DELETE triggers and historical identifiers.

- In a SQL client execute 07_demo.sql to demonstrate the views, function, procedure, SAVEPOINT and ROLLBACK.
- Show docs/evidence/database-results.json and integration-results.json. Explain constraints, failed-registration rollback, backend role tests and test cleanup.
- Explain individual contributions using actual commits and the report.

Reproduction

The database suite expects the original six-planet fixture. Run it before adding permanent demo planets. API tests create disposable records and remove them; audit history is intentionally preserved. Database setup preserves existing passwords and data.

Data dictionary

Generated from the running MySQL schema. All values and names below are inspected metadata.

app_account

Column	Type	Nullable	Default	Key	Extra
researcher_id	int	NO	NULL / none	PRI	-
password_hash	varchar(256)	NO	NULL / none	-	-
enabled	tinyint(1)	NO	1	-	-

- Foreign key: researcher_id -> researcher.researcher_id.
- Index: PRIMARY, column 1: researcher_id, BTREE, unique.

detection_method

Column	Type	Nullable	Default	Key	Extra
method_id	int	NO	NULL / none	PRI	auto_increment
method_name	varchar(80)	NO	NULL / none	UNI	-
description	text	NO	NULL / none	-	-

- Index: method_name, column 1: method_name, BTREE, unique.
- Index: PRIMARY, column 1: method_id, BTREE, unique.

discovery

Column	Type	Nullable	Default	Key	Extra
discovery_id	int	NO	NULL / none	PRI	auto_increment
planet_id	int	NO	NULL / none	MUL	-
mission_id	int	NO	NULL / none	MUL	-
method_id	int	NO	NULL / none	MUL	-
reported_by	int	NO	NULL / none	MUL	-
discovery_date	date	NO	NULL / none	MUL	-
signal_strength	decimal(12,4)	NO	NULL / none	-	-
notes	text	YES	NULL / none	MUL	-

- Foreign key: planet_id -> planet.planet_id.
- Foreign key: mission_id -> mission.mission_id.

- Foreign key: method_id -> detection_method.method_id.
- Foreign key: reported_by -> researcher.researcher_id.
- Check: (signal_strength > 0).
- Index: ft_discovery_notes, column 1: notes, FULLTEXT.
- Index: idx_discovery_date, column 1: discovery_date, BTREE.
- Index: method_id, column 1: method_id, BTREE.
- Index: mission_id, column 1: mission_id, BTREE.
- Index: planet_id, column 1: planet_id, BTREE, unique.
- Index: planet_id, column 2: mission_id, BTREE, unique.
- Index: planet_id, column 3: method_id, BTREE, unique.
- Index: planet_id, column 4: discovery_date, BTREE, unique.
- Index: PRIMARY, column 1: discovery_id, BTREE, unique.
- Index: reported_by, column 1: reported_by, BTREE.

mission

Column	Type	Nullable	Default	Key	Extra
mission_id	int	NO	NULL / none	PRI	auto_increment
mission_name	varchar(100)	NO	NULL / none	UNI	-
launch_date	date	NO	NULL / none	-	-
agency	varchar(100)	NO	NULL / none	-	-
status	varchar(20)	NO	NULL / none	-	-

- Check: (status in (_utf8mb4\'ACTIVE\',_utf8mb4\'COMPLETED\',_utf8mb4\'PLANNED\')).
- Index: mission_name, column 1: mission_name, BTREE, unique.
- Index: PRIMARY, column 1: mission_id, BTREE, unique.

mission_instrument

Column	Type	Nullable	Default	Key	Extra
instrument_id	int	NO	NULL / none	PRI	auto_increment
mission_id	int	NO	NULL / none	MUL	-
instrument_name	varchar(100)	NO	NULL / none	-	-
instrument_type	varchar(80)	NO	NULL / none	-	-

- Foreign key: mission_id -> mission.mission_id.

- Index: mission_id, column 1: mission_id, BTREE, unique.
- Index: mission_id, column 2: instrument_name, BTREE, unique.
- Index: PRIMARY, column 1: instrument_id, BTREE, unique.

mission_star

Column	Type	Nullable	Default	Key	Extra
mission_id	int	NO	NULL / none	PRI	-
star_id	int	NO	NULL / none	PRI	-

- Foreign key: mission_id -> mission.mission_id.
- Foreign key: star_id -> star.star_id.
- Index: PRIMARY, column 1: mission_id, BTREE, unique.
- Index: PRIMARY, column 2: star_id, BTREE, unique.
- Index: star_id, column 1: star_id, BTREE.

planet

Column	Type	Nullable	Default	Key	Extra
planet_id	int	NO	NULL / none	PRI	auto_increment
planet_name	varchar(100)	NO	NULL / none	UNI	-
star_id	int	NO	NULL / none	MUL	-
radius_earth	decimal(10,4)	NO	NULL / none	-	-
mass_earth	decimal(12,4)	NO	NULL / none	-	-
orbital_period_days	decimal(14,4)	NO	NULL / none	-	-
orbital_distance_au	decimal(12,5)	NO	NULL / none	-	-

- Foreign key: star_id -> star.star_id.
- Check: `(radius_earth > 0)`.
- Check: `(mass_earth > 0)`.
- Check: `(orbital_period_days > 0)`.
- Check: `(orbital_distance_au > 0)`.
- Index: planet_name, column 1: planet_name, BTREE, unique.
- Index: PRIMARY, column 1: planet_id, BTREE, unique.
- Index: star_id, column 1: star_id, BTREE.

researcher

Column	Type	Nullable	Default	Key	Extra
researcher_id	int	NO	NULL / none	PRI	auto_increment
first_name	varchar(60)	NO	NULL / none	-	-
last_name	varchar(60)	NO	NULL / none	-	-
email	varchar(254)	NO	NULL / none	UNI	-
role	varchar(20)	NO	RESEARCHER	-	-

- Check: `(role in (_utf8mb4\'ADMIN\',_utf8mb4\'RESEARCHER\'))`.
- Check: `regexp_like(email,_utf8mb4\'^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+[.][A-Za-z]{2,}$\')`.
- Index: email, column 1: email, BTREE, unique.
- Index: PRIMARY, column 1: researcher_id, BTREE, unique.

star

Column	Type	Nullable	Default	Key	Extra
star_id	int	NO	NULL / none	PRI	auto_increment
star_name	varchar(100)	NO	NULL / none	UNI	-
spectral_type	varchar(20)	NO	NULL / none	-	-
distance_ly	decimal(12,3)	NO	NULL / none	-	-
temperature_k	decimal(10,2)	NO	NULL / none	-	-
hz_inner_au	decimal(12,5)	NO	NULL / none	-	-
hz_outer_au	decimal(12,5)	NO	NULL / none	-	-

- Check: `(distance_ly > 0)`.
- Check: `(temperature_k > 0)`.
- Check: `(hz_inner_au > 0)`.
- Check: `(hz_outer_au > hz_inner_au)`.
- Index: PRIMARY, column 1: star_id, BTREE, unique.
- Index: star_name, column 1: star_name, BTREE, unique.

v_planet_catalogue

Column	Type	Nullable	Default	Key	Extra
planet_id	int	NO	0	-	-
planet_name	varchar(100)	NO	NULL / none	-	-
star_id	int	NO	NULL / none	-	-
radius_earth	decimal(10,4)	NO	NULL / none	-	-
mass_earth	decimal(12,4)	NO	NULL / none	-	-
orbital_period_days	decimal(14,4)	NO	NULL / none	-	-
orbital_distance_au	decimal(12,5)	NO	NULL / none	-	-
star_name	varchar(100)	NO	NULL / none	-	-
in_habitable_zone	int	NO	0	-	-

v_star_planet_count

Column	Type	Nullable	Default	Key	Extra
star_id	int	NO	0	-	-
star_name	varchar(100)	NO	NULL / none	-	-
planet_count	bigint	NO	0	-	-

validation

Column	Type	Nullable	Default	Key	Extra
researcher_id	int	NO	NULL / none	PRI	-
discovery_id	int	NO	NULL / none	PRI	-
validation_date	date	NO	NULL / none	-	-
confidence_level	decimal(5,2)	NO	NULL / none	-	-

- Foreign key: researcher_id -> researcher.researcher_id.
- Foreign key: discovery_id -> discovery.discovery_id.
- Check: `(confidence_level between 0 and 100)`.
- Index: idx_validation_discovery, column 1: discovery_id, BTREE.
- Index: idx_validation_discovery, column 2: confidence_level, BTREE.
- Index: PRIMARY, column 1: researcher_id, BTREE, unique.

- Index: PRIMARY, column 2: discovery_id, BTREE, unique.

validation_audit

Column	Type	Nullable	Default	Key	Extra
audit_id	bigint	NO	NULL / none	PRI	auto_increment
researcher_id	int	NO	NULL / none	-	-
discovery_id	int	NO	NULL / none	-	-
action	varchar(10)	NO	NULL / none	-	-
old_confidence	decimal(5,2)	YES	NULL / none	-	-
new_confidence	decimal(5,2)	YES	NULL / none	-	-
changed_at	timestamp(6)	NO	CURRENT_TIMESTAMP(6)	-	DEFAULT_GENERATED

- Index: PRIMARY, column 1: audit_id, BTREE, unique.

MongoDB notes

Document fields: `_id` (ObjectId), `discovery_id` and `author_id` (integers), `author_name`, `title`, `body` (strings), `tags` (array), `metadata` (nested object with source/status), `created_at` (BSON date).

Required types, string lengths and tags are enforced by the collection JSON schema. The API checks SQL discovery existence before inserting a note.

MongoDB indexes

```
[
  {
    "v": 2,
    "key": {
      "_id": 1
    },
    "name": "_id_"
  },
  {
    "v": 2,
    "key": {
      "_fts": "text",
      "_ftsx": 1
    },
    "name": "note_text",
    "weights": {
      "body": 1,
      "title": 3
    },
    "default_language": "english",
    "language_override": "language",
    "textIndexVersion": 3
  },
  {
    "v": 2,
    "key": {
      "discovery_id": 1,
      "created_at": -1
    },
    "name": "discovery_id_1_created_at_-1"
  }
]
```